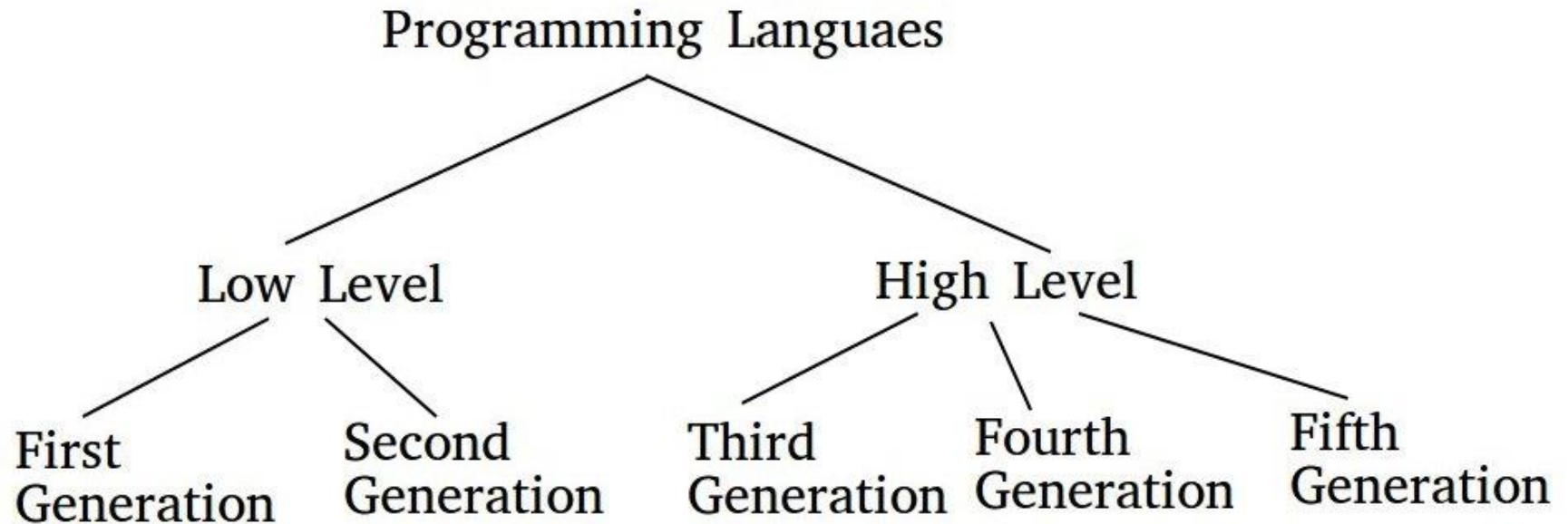


Generation and Evolution of Programming Languages

Lecture - 2

Generation of Programming Languages



First Generation Programming Languages

- Introduced in the 1940's
- Sometimes referred to as Binary Language, Machine Language, Very Low Level Language, Machine Code or Object Code
- It is a language made up of entirely 0's and 1's
- Programmers have to design their code by hand then transfer it to a computer by using a punch card, punch tape or flicking switches
- It is the only language a computer is capable of understanding without using a translation program
- Close to machines
- Modern day programmers still occasionally use machine level code, especially when programming lower level functions of the system, interface with firmware and hardware devices

First Generation Programming Languages

- Advantages...
 - Machine language makes fast and efficient use of the computer
 - It requires no translator to translate the code. It is directly understood by the computer
- Disadvantages...
 - All operation codes have to be remembered
 - All memory addresses have to be remembered
 - It is hard to amend or find errors in a program written in machine language

First Generation Programming Languages

- An example of machine language (binary) for the text “Hello World”

```
00100010 01101100 00110011 00110011  
01011101 00100000 01011101 01011101  
01011100 00110011 01001000
```

Second Generation Programming Languages

- Introduced in the 1950's
- Sometimes referred to as Assembly Language or Low Level Language
- Programs are in the form of Alphanumeric Symbols (or Mnemonic Codes) instead of 0's and 1's. These alphanumeric symbols can have maximum up to 5 letter combinations, e.g., ADD for addition, SUB for subtraction, etc. Because of this feature it is also known as "Symbolic Programming Language"
- Close to machines
- Programs are translated into machine language using Assemblers
- Not portable
- Used in kernel and hardware driver, but more often used in extremely intensive processing such as games, video editing, graphic manipulation/rendering

Second Generation Programming Languages

- Advantages...
 - Assembly language is easier to understand and use as compared to machine language
 - It is easy to locate and correct errors
- Disadvantages...
 - Like machine language, it is also machine dependent/specific
 - Since it is machine dependent, the programmer also needs to understand the hardware

Second Generation Programming Languages

- An example of assembly language for the text “Hello World”

```
section .text
    global _start    ;must be declared for linker (ld)

_start:              ;tells linker entry point
    mov     edx,len   ;message length
    mov     ecx,msg    ;message to write
    mov     ebx,1      ;file descriptor (stdout)
    mov     eax,4      ;system call number (sys_write)
    int     0x80       ;call kernel

    mov     eax,1      ;system call number (sys_exit)
    int     0x80       ;call kernel

section .data
msg db 'Hello, world!', 0xa ;string to be printed
len equ $ - msg           ;length of the string
```


Second Generation Programming Languages

- Examples which are commonly in use today are...
 - RISC (Reduced instruction set computer)
 - CISC (Complex Instruction Set Computing)
 - x86 as that is what our embedded systems and desktop computers use

Third Generation Programming Languages

- Introduced in the 1950's
- Sometimes referred to as High Level Languages (HLL)
- Purpose of developing HLLs was to enable people to write programs easily, in their native language environment (English)
- Close to humans
- Compiler (which converts the language into machine code automatically) was developed
- First compiled high level language: Autocode in 1952 for the Mark 1 computer at the University of Manchester
- In 1954, Fortran was invented at IBM by John Backus. It was the first widely used high level general purpose programming language to have a functional implementation.

Third Generation Programming Languages

- Advantages...
 - Easier to learn and understand than assembly language
 - Have less rigid rules, forms and syntax, so the potential for error is reduced
 - Programs written are machine independent
 - Programmers do not have to learn a new language for each computer they program
- Disadvantages...
 - Less efficient than assembly language programs and require a great amount of computer time for translation into machine instruction.

Types of High Level Languages

1. Algebraic Formula-Type Processing

- These languages are oriented towards the computational procedures for solving mathematical and statistical problems
- Examples:
 - BASIC (Beginners' All-purpose Symbolic Instruction Code)
 - FORTRAN (Formula Translation)
 - PL/I (Programming Language One)
 - ALGOL (Algorithmic language)
 - APL (A Programming Language)

2. Business Data Processing

- These languages best able to maintain data processing features and problems involved in handling files
- Examples:
 - COBOL (Common Business Oriented Language)
 - RPG (Report Program Generator)

Types of High Level Languages

3. String and List Processing

- These are used for string manipulation, including searching patterns and inserting/deleting characters
- Examples:
 - LISP (List Processing)
 - Prolog (Programming in Logic)

4. Object-oriented Programming Languages

- In OOP, the computer program is divided into objects.
- Approx 50 types
- Examples:
 - Object C
 - C++
 - Java
 - C#

Types of High Level Languages

5. Scripting Languages

- These languages are used where instructions are written for a run time environment
- They do not require the compilation step and are rather interpreted
- Examples:
 - Ruby
 - Python
 - Perl

Fourth Generation Programming Languages

- 1970's through 1990's
- Also known as Very High Level or Non-procedural Language
- It is application specific
- Close to natural language
- Need approx. one tenth the number of statements that a high level language need to achieve the same result
- Non-computer professionals can develop the software
- Examples
 - Query languages (SQL)
 - Report Program Generator (RPG by IBM)
 - Data management languages (SAS, SAAS, Sata)
 - Matrix Laboratory (MATLAB)

Fifth Generation Programming Languages

- It is based on solving using constraints given to the computer rather than using an algorithm written by the programmer
- Introduced around 1990's
- Very closely resembles human speech
- Examples:
 - Prolog, Mercury, OPS5, ICAD, etc.
- These languages are designed to make the computer “smarter”
- Mainly used in artificial intelligence research
- Natural languages already available for microcomputers include Clout, Q&A and Savvy Retriever (for use with database) and HAL (Human Access Language)

History of Programming Languages

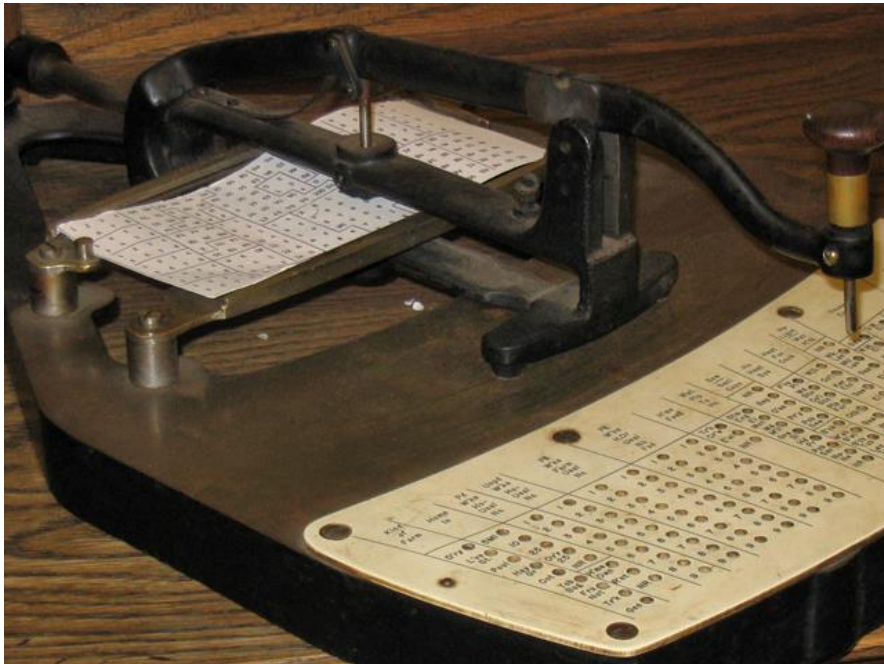
1842-1843

- **First computer program created by Ada Lovelace**
- Work on the Charles Babbage's analytical engine. A complete detail a method for calculating Bernoulli numbers with the Engine.



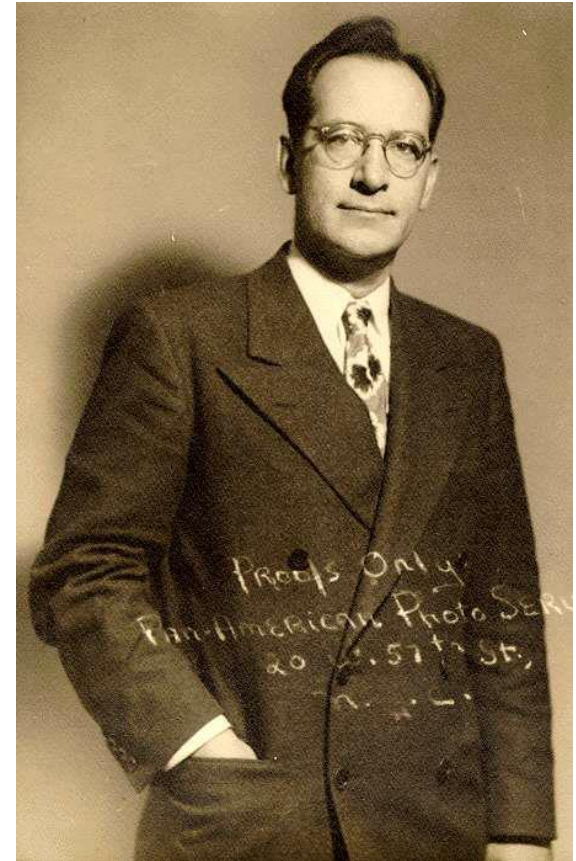
1890

- Census data on punch cards encoded by **HERMAN HOLLERITH**



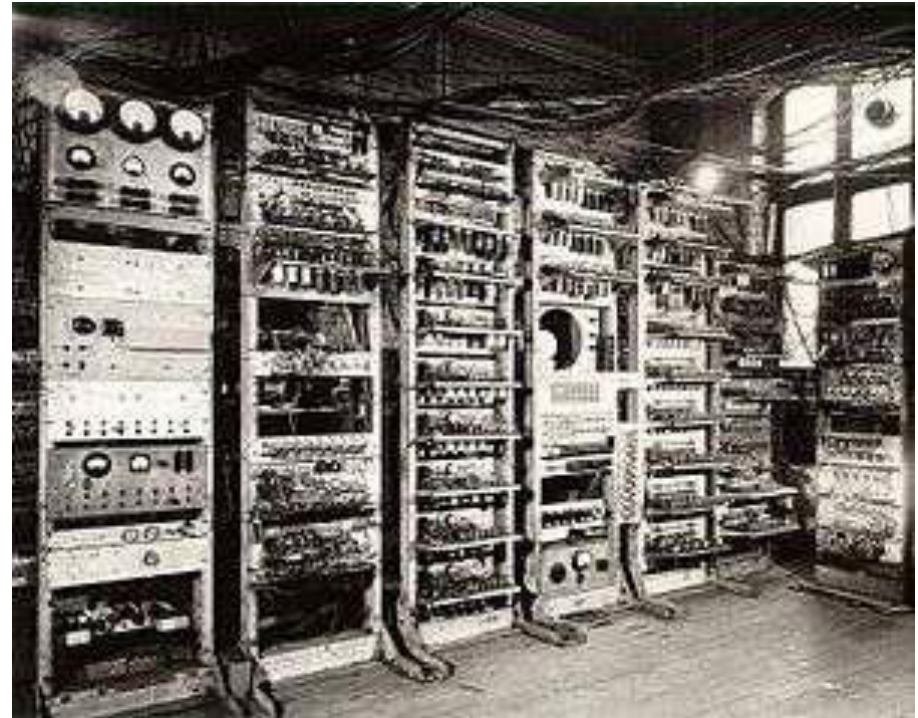
1949

- **Short code develop by JOHN MAUCHLY**
- One of the first high-level languages ever developed for an electronic computer.
- Short code statements are represented by mathematical expressions.



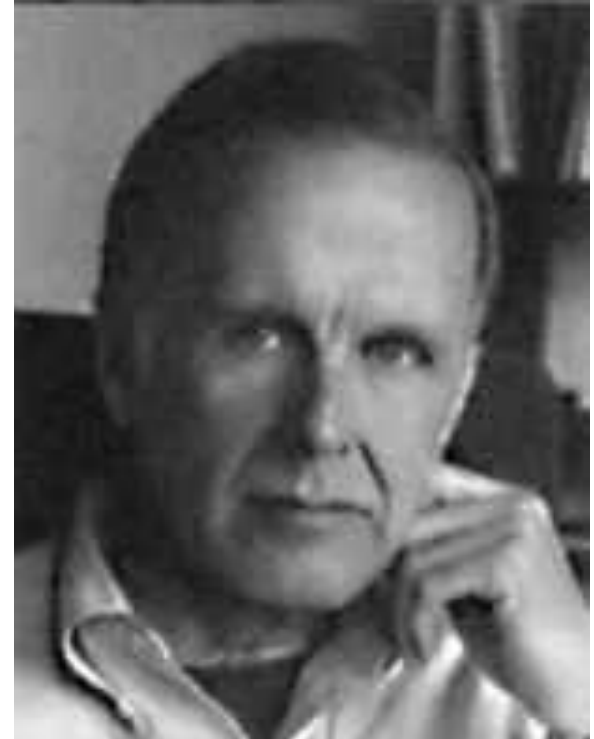
1952

- **Manchester Mark 1 developed by ALICK GLENNIE**
- Considered as first compiled high – level programming language.
- Automatically convert language into a machine code.



1957

- **Fortran created by JOHN BACKUS**
- For numeric and scientific computing (as an alternative to assembly language)
- Oldest programming language used today (NASA)

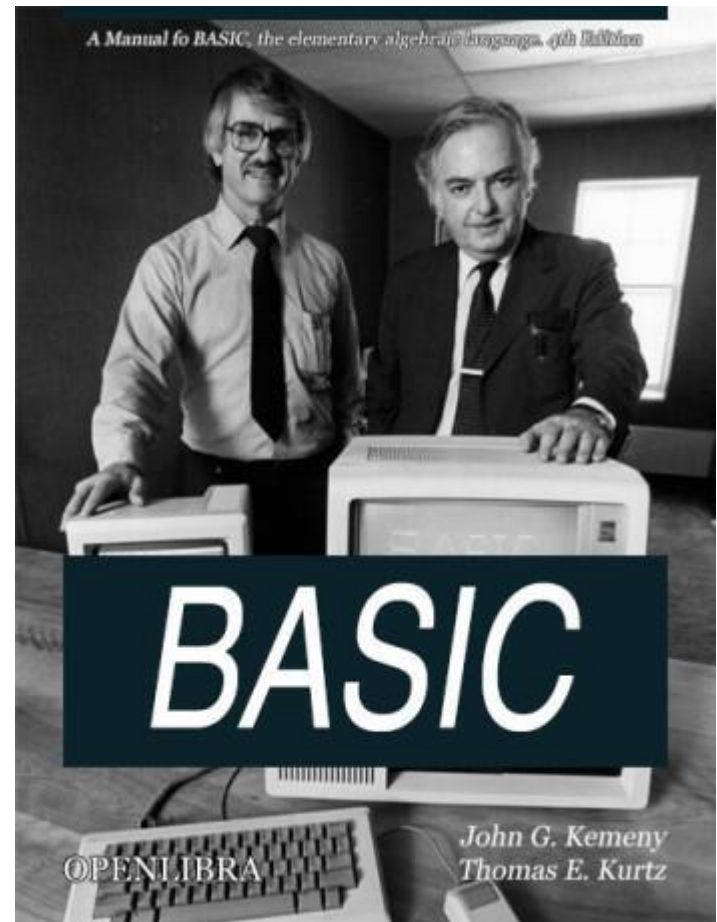


1959

- **COMMON BUSINESS ORIENTED LANGUAGE (COBOL)** created by Short Range committee.
- High-level. Primarily for business computing. First programming language to be mandated by the US Department of Defense.
- Primary Uses: Business software (esp. finance and administration systems, but also banks, insurance agencies, governments, military agencies)
- Used By: Credit cards, ATMs
- **Fun Fact* Action movie The Terminator used samples of Cobol source code for the text shown in the Terminator's vision display

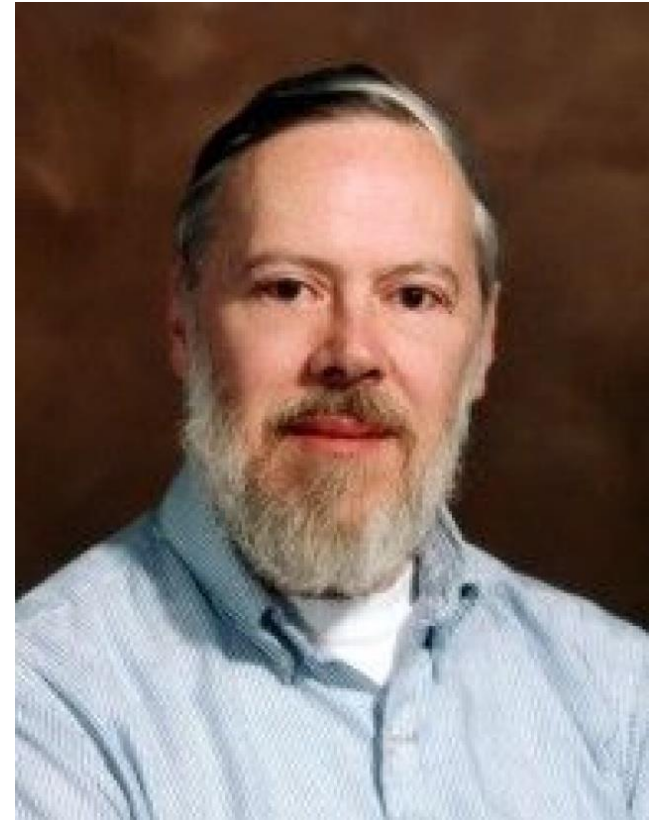
1964

- **BASIC (BEGINNER'S ALL-PURPOSE SYMBOLIC INSTRUCTION CODE)** created by **JOHN GEORGE KENNY** and **THOMAS EUGENE KURTZ**
- Designed for simplicity.
- Computer games were often written in Basic.(Microsoft's Altair, Apple II)



1972

- **C created by DENNIS RITCHIE of BELL LABS**
- Currently the world's most popular programming language
- Many leading languages are derivatives, including C#, Java, JavaScript, Perl, PHP, and Python
- System programming, computer game development.(UNIX)



1980

- **ADA created by JEAN ICHBIAH**
- Derived from Pascal.
- For developing large software systems.(NASA,NSTAR)
- Primary Uses: Dept. of Defense, banking, manufacturing, transportation, commercial aviation



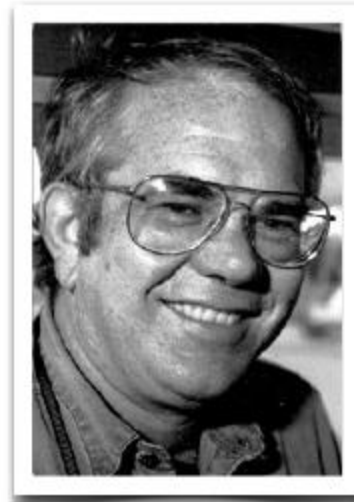
1983

- **C++ created by BJARNE STROUSTRUP**
- An extension of C, with enhancements such as classes, virtual functions, and templates.
- Primary Uses: Commercial application development, embedded software, server/client applications, video games
- Used By: Adobe, Google Chrome, Mozilla Firefox, Microsoft Internet Explorer



1983

- **Objective-C created by BRAD COX and TOME LOVE**
- Expanded on C, adding message-passing functionality based on Smalltalk language.
- Used By: Apple's OS X and iOS operating systems



1991

- **Python created by GUIDO VAN ROSSUM**
- Created to support a variety of Programming styles and be fun to use.
- Primary Uses: Web application, software development, information security
- Google, Yahoo, Spotify



1993

- **Ruby created by YUKIHIRO MATSUMOTO**
- Designed for productive and enjoyable programming (Twitter, Hulu, Groupon)
- Primary Uses: Web application development



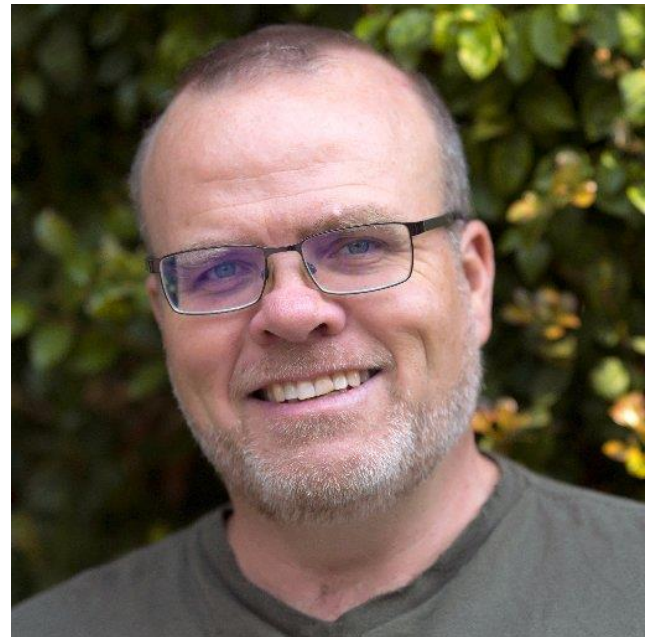
1994

- **Java created by JAMES GOSLING**
- Made for an interactive TV project. Cross-platform functionality. Second most popular language (behind C).(Android OS/apps)
- Primary Uses: Network programming, web application development, software development, Graphical User Interface development



1995

- **PHP (Personal Home Page) created by RASMUS LERDORF**
- For building dynamic web pages. Most widely used open-source software by enterprises.
- Primary Uses: Building/maintaining dynamic web pages, server-side development
- Used By: Facebook, Wikipedia, Digg, WordPress, Joomla



1996

- **JavaScript created by BRENDAN EICH**
- Created to extend web page functionality.
- Dynamic web pages use for form submission/validation, interactivity, animations, user activity tracking, etc.
- Primary Uses: Dynamic web development, PDF documents, web browsers, desktop widgets
- Used By: Gmail, Adobe Photoshop, Mozilla Firefox

